Universidad Nacional de Río Cuarto – UNRC
Base de Datos

INFORME DE PROGRESO DE PROYECTO

Diagrama E/R, Tabulación, DDL,DML





Integrantes:

- Fassano Lucio
- Maldonado Román
- Rios Ticiano
- Saliba Maximo
- Tello Valentin

1. Introducción al problema y el objetivo

El presente informe abordará la solución a la problemática presentada a los integrantes del curso de segundo año de la materia “Bases de datos 1” con el fin de aplicar los conocimientos teórico-prácticos adquiridos durante el cursado de la materia, es por ello que como primera medida del informe se presenta una breve descripción del problema a resolver:

La problemática se presenta mediante un enunciado desarrollado en un texto, el cual dictaba una situación sobre una zapatería, más específicamente la tienda/empresa “Calzados S.A”, la cual requería del diseño y construcción de una base de datos para la gestión de sus ventas, esta base de datos debía de contener y gestionar la información de vendedores, productos que están disponibles para la venta y que fueron vendidos, proveedores de los productos, clientes que realizaron una compra, entre varias cuestiones más.

Con el problema sobre la mesa ya podemos introducir el objetivo del equipo en la primera etapa de este proyecto. El objetivo en esta etapa era el análisis del problema para la realización de un modelo Entidad Relación y una de tabulación dicho modelo que cumpliera con todos los requisitos del enunciado.

2. Diseño Conceptual: Modelo Entidad-Relación

A continuación, se presenta toda la información del desarrollo del modelo entidad Relación.

2.1. Decisiones durante el diseño

Se asume que el monto de la entidad venta realizada a un cliente es un atributo derivado que se calcula a partir del precio, al momento de la venta, y cantidad de los productos participantes en la venta y del descuento que el cliente posee

Se asume que el atributo Observaciones de la entidad Franco se utiliza para registrar información excepcional, aclaraciones o detalles contextuales que no pueden describirse en los demás campos de la entidad.

Se asume que el precio de los zapatos podría variar según el número de talle, modelo, y color del zapato, es por ello que el precio de el zapato no se encuentra en la clase Zapato.

Al atributo celular existente en la entidad proveedor optó por diagramarlo como un atributo multivaluado debido a que el anunciado lo expresaba como “celulares”.

La Relación realiza entre las entidades cliente y venta posee el porcentaje de descuento como atributo debido a que en un futuro un minorista podría recibir un descuento especial debido a una situación especial, como también a un cliente mayorista podría sucederle esta situación además sumándole al descuento que ya posee. Además por este momento consideramos que este atributo el valor que se aplica al atributo calculado monto.

Asumimos que un mismo zapato puede tener varios proveedores.

Una vez pudimos plantear y dejar expresadas estas consideraciones antes de la realización del diagrama pudimos comenzar con el desarrollo del modelo.

2.2. Diccionario de Entidades y Relaciones

Como una forma de otorgar más información del modelo entidad Relación hemos decidido dar una breve explicación de cada entidad y de las Relaciones entre ellas, junto con sus cardinalidades:

2.2.1. Entidades y Atributos

Zapato: Entidad pilar del sistema, representa al conjunto de zapatos existentes dentro del sistema que están disponibles mediante la venta, cada Zapato se identifica inequívocamente mediante su color y modelo.

Proveedor: Entidad que representa a los distintos proveedores de zapatos que alguna vez solventaron de stock a la zapatería, se identifica inequívocamente por su atributo CUIT, posee atributos añaden información extra, como sus celulares que como lo dijimos anteriormente era un atributo multivaluado.

Talle: Entidad la cual representa el talle de una zapatilla. Esta entidad está relacionada con zapato de manera que, **las zapatillas tienen N cantidad de N talles, los cuales tienen una cantidad de pares y un precio determinado.**

Venta: Entidad más importante dentro del sistema, representa una venta efectuada a un cliente por la zapatería mediante un vendedor, posee atributos como el número de factura nro_factura que permite identificar inequívocamente a cualquier venta, posee información del cliente que realizó la compra, el vendedor que efectuó la venta y el monto de la venta, siendo este último un atributo multivaluado que se

calcula a partir del descuento del cliente y cantidad y precio de los zapatos que el cliente compró.

Cliente, Mayorista, Minorista: Estas tres entidades representan una generalización, cliente posee la Relación principal y los atributos más importantes como la clave primaria dni, mayorista que representa a un comprador al por mayor de la empresa, posee el atributo descuento_general atributo que representa el descuento que recibe siempre que realiza una compra y que afecta al atributo porcentaje_descuento de la Relación realiza, y luego tenemos a Minorista que representa a un comprador común de la tienda donde almacenamos además de todos los otros datos su fecha de nacimiento.

Vendedor: Entidad que representa a un vendedor de la tienda, posee todos sus datos personales, pero principalmente posee su atributo dni_vendedor que permite identificar inequívocamente a cada vendedor.

Dia: Entidad que representa los días de la semana. Utilizada para expresar el horario de entrada y salida de un vendedor. Se identifican inequívocamente mediante el atributo nombre_dia, representa en nombre del día de la semana.

Franco: Entidad que representa las fechas y motivos del franco de un vendedor, se identifican inequívocamente mediante el nro_franco.

2.2.2. Relaciones y Cardinalidades

***provee*:** Relación entre proveedor y zapato, con cardinalidad N:M, representa los productos que los distintos proveedores solventaron a la zapatería.

***tiene_talle*:** Relación entre Zapato y Talle, con cardinalidad N:M permitiendo representar el talle, precio y stock disponible de un modelo y color de un zapato, mediante los atributos de la relación precio y cantidad

contiene: Relación entre la agregación (Talle, Zapato) y Venta, con cardinalidad N:M que representa los zapatos participantes dentro de una venta con los atributos precio_venta que representa el precio actual por el que el zapato está vendiéndose y cantidad_venta cantidad de ese zapato en específico.

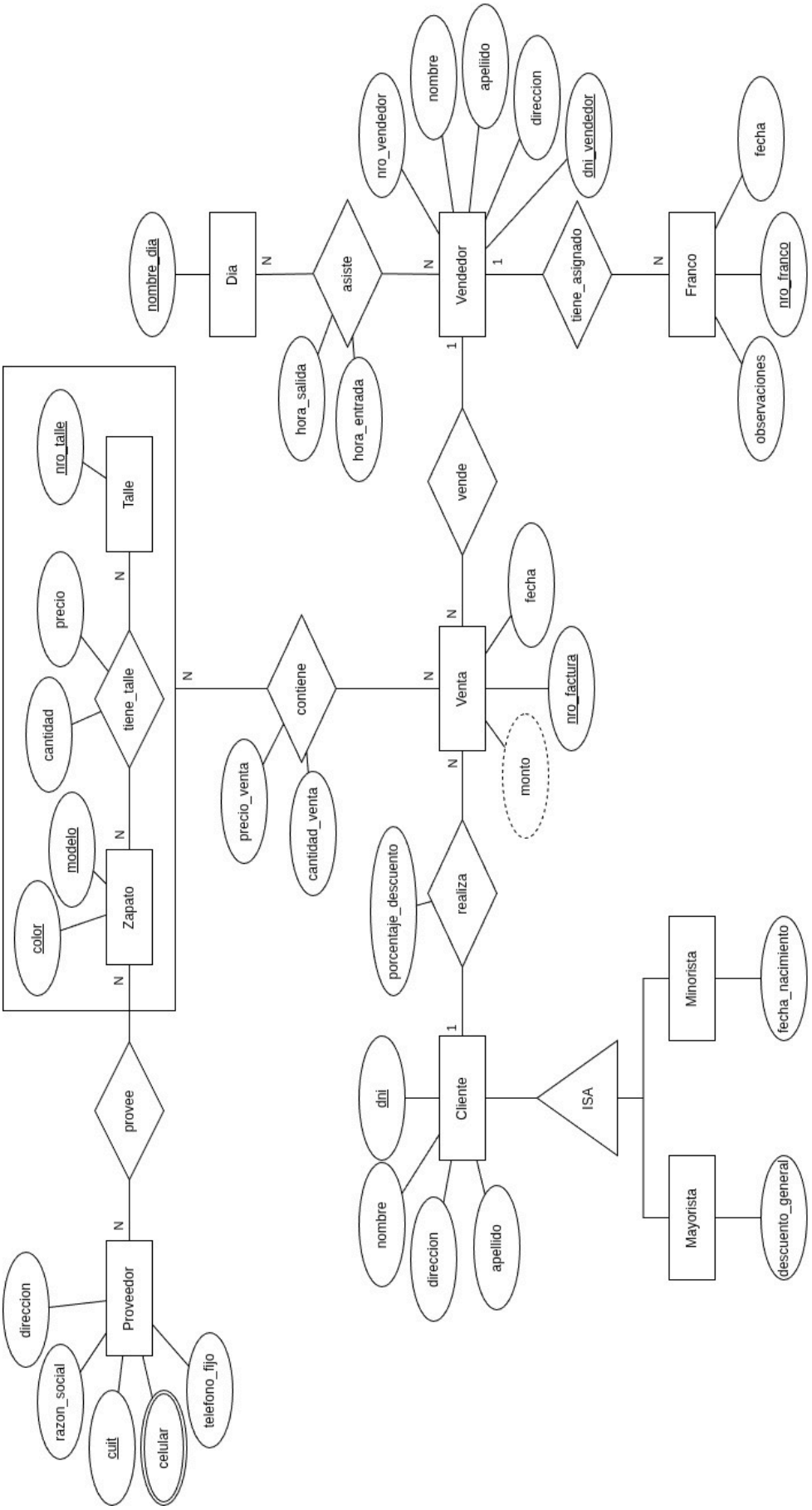
realiza: Relación entre Cliente y Venta, con cardinalidad 1:N que representa la realización de una venta por parte de la zapatería o la compra de un cliente a la zapatería, posee el atributo porcentaje_descuento que es el descuento que se le aplica el monto total de la venta/compra.

vende: Relación entre Venta y Vendedor, con cardinalidad N:1, representando la participación de un vendedor dentro de una venta, que puede participar en ninguna o varias ventas y que una venta necesita exclusivamente un vendedor.

asiste: Relación entre Día y Vendedor, con cardinalidad N:M, representa el horario de ingreso y salida de un vendedor en los días de la semana en los que ejerce su labor, posee los atributos hora_salida y hora_entrada, representando el horario de entrada y salida del trabajador en un día en específico.

tiene_asignado: Relación entre Vendedor y Franco, con cardinalidad 1:N que representa la Relación entre un vendedor y sus fechas de franco, donde una entidad Vendedor puede tener varios francos y una entidad Franco puede tener exclusivamente una única entidad Vendedor asociada.

Jerarquia de Cliente: generalización entre la entidad padre Cliente y las entidades hijas Mayorista y Minorista, con carácter total-disjunta ya que un cliente siempre es un Mayorista o un Minorista pero a la vez debe de ser únicamente uno de los dos tipos, donde estos tipos heredan todos los atributos de cliente y poseen sus atributos específicos.


nombre_dia
Dia
N
asiste
N
Vendedor
1
tiene_asignado
N
Franco
nro_vendedor
nombre
apellido
direccion
dni_vendedor
fecha
nro_franco
observaciones
hora_salida
hora_entrada
1
vende
N
Venta
fecha
nro_factura
monto
N
realiza
1
Cliente
dni
nombre
direccion
apellido
ISA
Minorista
fecha_nacimiento
Mayorista
descuento_general
porcentaje_descuento
N
contiene
N
precio_venta
cantidad_venta
N
tiene_talle
precio
cantidad
N
Talle
nro_talle
N
Zapato
modelo
color
N
provee
N
Proveedor
direccion
razon_social
cuit
celular
telefono_fijo

3. Tabulación

3.1. Estructura Sección de Notación

La tabulación del diagrama está dividida en varias secciones, siendo ellas:

- Entidades Fuertes, Débiles y Atributos multivaluados.
- Jerarquía de Clientes.
- Relaciones y Entidades Asociadas a eventos.

Dentro de **Entidades Fuertes, Débiles y Atributos multivaluados** se encontrarán las tabulaciones de las entidades Proveedor, MCelular, el cual es un atributo multivaluado de proveedor, Vendedor, Zapato, Talle y Dia. Cada tabulación está acompañada de su Clave Primaria subrayada y cada vez que se utiliza está en otra entidad se especifica el uso de la misma.

Dentro de **Jerarquía de Clientes** se encontrarán las tabulaciones relacionadas a la generalización de clientes, dependiendo el caso que haya que usar.

Y por último, dentro de **Relaciones y Entidades Asociadas a eventos** se encontrarán las tabulaciones de las relaciones N:M y de la relación de agregación presentada en el diagrama E/R.

3.2. Tabulación final del Diagrama

3.2.1. Entidades Fuertes, Débiles y Atributos multivaluados

- Franco(nro_franco, observaciones, fecha, dni_vendedor)
  - nro_franco es **Claves Primaria** de Franco.
  - dni_vendedor es **Clave Foránea** que hace referencia a Vendedor.

- Vendedor(dni_vendedor, nombre, apellido, direccion, nro_vendedor)
  - dni_vendedor es **Claves Primaria** de Vendedor.

- Dia(nombre_dia)
  - nombre_dia es **Claves Primaria** de Dia.

- Proveedor(cuit, razon_social, direccion, telefono_fijo)

- Mcelular(cuit, celular)

- cuit es **Clave Foránea** que hace referencia a Proveedor.

- Zapato(color, modelo)
  - color, modelo son **Claves Primarias** de Zapato.

- Talle(nro_talle)
  - nro_talle es **Claves Primaria** de Talle.

- Venta(nro_factura, monto, fecha, dni, dni_vendedor, porcentaje_descuento)
  - nro_factura es **Claves Primaria** de Venta.
  - dni es **Clave Foránea** que hace referencia a Cliente.
  - dni_vendedor es **Clave Foránea** que hace referencia a Vendedor.

3.2.2. Jerarquía de clientes

- Cliente(dni, nombre, dirección, apellido)
  - dni es **Claves Primaria** de Cliente.

- Mayorista(dni, descuento_general)
  - dni es **Clave Foránea** que hace referencia a Cliente.

- Minorista(dni, fecha_nacimiento)
  - dni es **Clave Foránea** que hace referencia a Cliente.

3.2.3. Relaciones

- tiene_talle(color, modelo, nro_talle, cantidad, precio)
  - color, modelo son **Clave Foránea** que hace referencia a Zapato.
  - nro_talle es **Clave Foránea** que hace referencia a Talle.

- contiene(color, modelo, nro_talle, nro_factura, precio_venta, cantidad_venta)
  - color, modelo son **Clave Foránea** que hace referencia a Zapato.
  - nro_talle son **Clave Foránea** que hace referencia a Talle.
  - nro_factura es **Clave Foránea** que hace referencia a Venta.

- asiste(dni_vendedor, nombre_dia, hora_entrada, hora_salida)
  - dni_vendedor es **Clave Foránea** que hace referencia a Vendedor.
  - nombre_dia es **Clave Foránea** que hace referencia a Dia.

- provee(cuit, color, modelo)
  - color, modelo son **Clave Foránea** que hace referencia a Zapato.
  - cuit es **Clave Foránea** que hace referencia a Proveedor.

4. Base de Datos: Implementar Tablas

Para esta etapa del proyecto, lo que hicimos fue implementar todas las tablas resultantes de la tabulación del diagrama realizado con anterioridad en un script SQL, implementado/modificado dentro del motor **MySQL Workbench.**

4.1. ¿Por qué elegimos MySQL y no PostgreSQL?

Utilizamos MySQL porque, aunque la sintaxis entre MySQL y PostgreSQL sea similar, se nos hizo mucho más fácil a la hora de crear las tablas e insertar los datos de las mismas. En los casos que fue más fácil comparar el funcionamiento de MySQL y PostgreSQL fue cuando tuvimos que **diseñar el trigger de la tabla auditoría** y **diseñar la generalización de cliente,** que si bien, en ambos motores, el trigger funciona igual, para nosotros, fue mucho más fácil entender la sintaxis entender la sintaxis del **trigger creado con MySQL** que la del **trigger creado con PostgreSQL.** Con la **generalización** fue un caso similar, ya que la generalización en PostgreSQL es similar a este ejemplo:

```sql
-- Tabla padre (Generalización)
CREATE TABLE empleado (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    salario NUMERIC
);

-- Tabla hija (Especialización)
CREATE TABLE gerente (
    bono_anual NUMERIC,
    departamento VARCHAR(50)
) INHERITS (empleado);
```

En donde entendemos que con la función INHERITS() lo que hacemos es decirle a una tabla que es tabla hija, o sea, que es heredada a partir de una tabla padre.

A la hora de implementar la generalización en MySQL, nos pareció mucho más significativo representarla mediante el uso de claves primarias y claves foráneas, ya que, es como lo vimos representado en la tabulación realizada sobre el diagrama planteado con anterioridad. No afirmamos que MySQL sea mejor que PostgreSQL, simplemente

elegimos MySQL por su sencillez a la hora de entender algunas porciones de código que conllevaba la creación de nuestras tablas.

4.2. Información sobre la implementación de las tablas.

A continuación, vamos a explicar lo que implementamos dentro del código, decisiones de equipo, bloques de código, etc. Todo va a estar numerado, de manera ascendente (desde la línea 1 hasta la línea N).

1. **DROP DATABASE IF EXISTS TPBDD:**
   Lo que hacemos con esta porción de código es preguntarnos si efectivamente **existe una base de datos ya creada con anterioridad**, en el caso de que exista, simplemente hacemos un DROP DATABASE y creamos una base de datos nueva y limpia, esto lo hacemos de manera tal que el proyecto siempre se limpie de manera automática cada vez que ejecutamos nuestro script. Obviamente, en la práctica profesional no se debería de hacer esto, nosotros lo decidimos hacer para que cada vez que se desee probar la base, sea de manera clara, limpia y concisa.

2. **DROP TABLE IF EXISTS TABLE X:**
   Este comando funciona igual que el anterior, pero, ahora en vez de limpiar la base de datos, limpiamos todas las tablas de la misma. En caso de que exista una tabla **“x”**, ejecutamos DROP TABLE IF EXISTS X para crear una nueva tabla limpia, más adelante le cargaremos datos a la misma.

3. **Tablas relacionadas a entidades**:
   A la hora de implementar las tablas relacionadas con las entidades, lo que hicimos fue simple, simplemente creamos la tabla con los atributos tabulados del diagrama e/r, agregamos el tipo que conlleva cada atributo y luego, mediante la cláusula **CONSTRAINT** designamos cierto tipo de restricciones. Cada vez que le decimos a una tabla, cuál es su **clave primaria**, lo hacemos mediante **CONSTRAINT pk_atributo PRIMARY KEY (atributo)** de tal manera que cada tabla tiene su y/o sus claves primarias.

4. **FOREIGN KEY:**
   A la hora de decirle a una tabla que un atributo de la misma es una **clave foránea que referencia a otra tabla** lo hacemos mediante el comando **CONSTRAINT fk_tablaVecina FOREIGN KEY (atributo) REFERENCES tablaVecina(atributo)**, de esta manera, le decimos a nuestro motor que el atributo que está por procesar, es una **clave foránea que hace referencia a “x” tabla.**

4.3. Información sobre la implementación de las restricciones.

Para poder explicar cómo implementamos las restricciones pedidas por el ejercicio, vamos a enumerar cada una, fundamentar cada una y mostrar una porción de código de la misma.

1. ***"Los números de los vendedores deberán ser generados"***
   En una primera versión relacionada con esta restricción tuvimos el siguiente problema. Leyendo en internet, encontramos una función predefinida de MySQL llamada **AUTO_INCREMENT** que básicamente incrementa un atributo definido en una tabla. Nos pareció una buena idea, pero, a la hora de insertar datos vino el problema, ya que, no sabíamos cómo insertar **nro_vendedor** en relaciones que usarán nro_vendedor como clave foránea, buscamos por internet y encontramos soluciones relacionadas con subconsultas o variables locales, haciendo que de esta manera el código no sea tan eficiente y sea muy difícil de entender para nosotros. Por ende, tuvimos que cambiar un poco nuestro modelado del problema, haciendo que **dni_vendedor** sea la **clave primaria** de la tabla Vendedor, ya que, usando dni_vendedor como clave foránea en relaciones que la usen, no tendríamos problemas con código que no razonemos.

   Es por eso que, a la hora de implementar esta restricción, tuvimos que rediseñar nuestro razonamiento acerca del dominio del problema, estableciendo **dni_vendedor** como **clave primaria de la tabla Vendedor.**

   Para implementar esta restricción, lo que hicimos fue asignarle la función **AUTO_INCREMENT** a **nro_vendedor**, la cual incrementa de manera automática el valor entero.

```
CREATE TABLE vendedor(
    dni_vendedor INT NOT NULL,
    nro_vendedor INT AUTO_INCREMENT NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    direccion VARCHAR(100) NOT NULL,

    CONSTRAINT pk_vendedor PRIMARY KEY(dni_vendedor),
    UNIQUE (nro_vendedor)
);
```

   Luego, le establecemos que **dni_vendedor** es la **clave primaria de nuestra tabla.** Luego, establecemos la ligadura de integridad **UNIQUE** a **nro_vendedor,** para que de esta manera, no haya números autogenerados **que sean iguales.**

2. ***"El nùmero de los zapatos es un número positivo menor a $200.000"***
   Para representar esta restricción dentro de nuestra base de datos, lo que hicimos fue que, a la hora de crear la tabla **contiene** la cual, es aquella que representa a la relación de agregación diagramada con anterioridad, comprobamos con una cláusula **CONSTRAINT y CHECK** si efectivamente el atributo de la relación **precio_venta** es menor a 200.000.

   Codigo: **CONSTRAINT check_precio CHECK (precio_venta < 200000)**

3. ***"El descuento, tanto el gral de los mayoristas, como el particular de cada venta, es un número real en 0 y 100"***
   Muy similar a lo planteado anteriormente, dentro de la tabla venta, creamos un atributo llamado **porcentaje_descuento**, el cual es un atributo de la relación entre la entidad Venta y la entidad Cliente. A este atributo lo chequeamos con las cláusulas **CONSTRAINT** y **CHECK,** de tal manera que comprobamos si el porcentaje es mayor o igual a 0 y menor a 100.

   Para el descuento de los mayoristas, aplicamos el mismo código pero cambiando los atributos.

   Códigos:
   - **CONSTRAINT check_descuento1 CHECK (porcentaje_descuento >= 0 AND porcentaje_descuento < 100)**
   - **CONSTRAINT check_descuento2 CHECK (descuento_general >= 0 AND descuento_general < 100)**

4.4. Información sobre la implementación del Trigger.

Para implementar la restricción ***"La implementación de la base de datos deberá permitir generar información de auditoría automáticamente. Se deberá agregar información en una tabla sobre los cambios de precio en los zapatos, está información deberá contener, fecha, precio anterior, precio nuevo y usuario que realizó el cambio."*** decidimos implementar un Trigger.

```
DELIMITER $$

CREATE TRIGGER trigger_auditoria
AFTER UPDATE ON tiene_talle
FOR EACH ROW
BEGIN
    IF OLD.precio <> NEW.precio THEN
        INSERT INTO auditoria_zapato_precio(fecha, precio_viejo, precio_nuevo, usuario)
        VALUES(NOW(), OLD.precio, NEW.precio, USER());
    END IF;
END$$

DELIMITER ;
```

Primero comenzamos delimitando nuestro trigger con DELIMITER, esto para que esté separado de los bloques que crean las tablas. Luego, con **AFTER UPDATE on tiene_talle**, le decimos que **después de que se actualice un producto de la tabla tiene_talle haga lo que tiene dentro de BEGIN.** El bloque BEGIN nos ofrece dos variables temporales, una llamada **OLD** y la otra llamada **NEW**. OLD contiene aquellos datos los cuales son almacenados dentro de la variable **ANTES de que se ejecute la operación, en este caso el AFTER UPDATE**, y **NEW contiene los datos actualizados.**
Dicho esto, nosotros comparamos los precios, si OLD.precio es distinto de NEW.precio, entonces lo insertamos a nuestra **tabla auditoria.** La tabla auditoría, contiene campos como fecha, precio_viejo, precio_nuevo, usuario. La fecha la calculamos con una función predefinida llamada **NOW(),** al precio viejo le asignamos lo que tenga **OLD.precio**, al precio nuevo le asignamos lo que tenga **NEW.precio** y a usuario, le asignamos lo que retorne la función predefinida **USER().**

4.5. Información sobre las restricciones de FOREING KEY.

A la hora de usar un atributo como clave foránea, es decir, representa a otra tabla, cada vez que borramos o actualizamos el mismo, se efectúa un cambio en cascada, esto lo decidimos así para que no haya problemas de concurrencia.

5. Base de datos: Insertar Datos

A la hora de insertar los datos dentro de la base, nos basamos en datos de la vida real, como por ejemplo nuestros nombres y apellidos. Simplemente lo que hicimos fue usar la cláusula **INSERT** de manera repetida con todas las tablas para cargar sus respectivos datos.

6. Datos curiosos transcurridos durante el desarrollo.

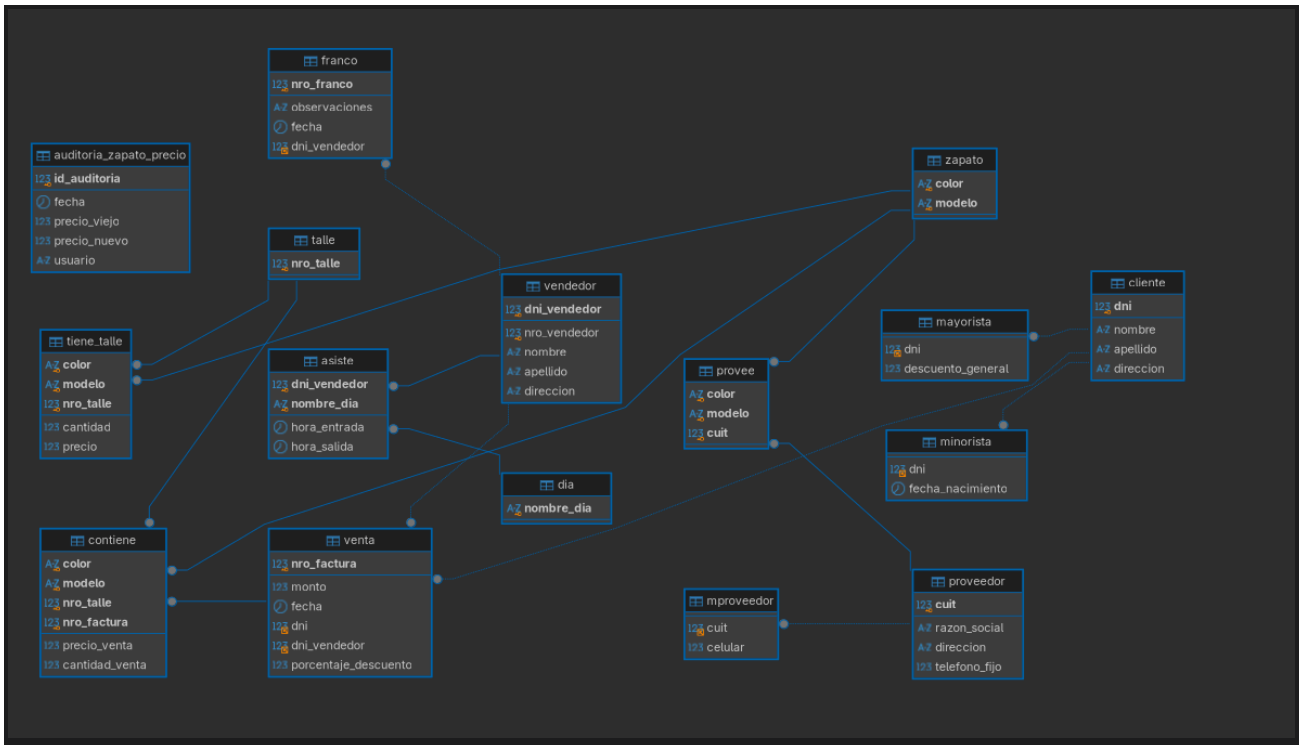
6.1. DBeaver

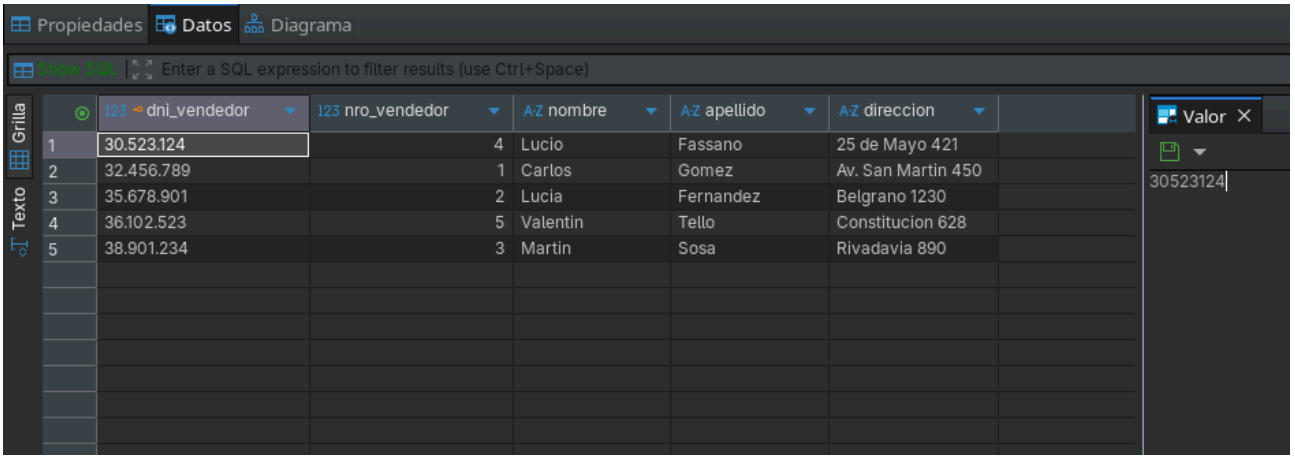


A la hora de desarrollar la base de datos, desarrollamos gran parte de la base con una herramienta llamada **DBeaver.** Según la definición de Wikipedia: DBeaver es una

herramienta de administración de bases de datos universal y cliente SQL de código abierto, basada en la plataforma Eclipse y escrita en Java. Su principal característica es permitir la gestión de múltiples tipos de bases de datos (más de 70 motores) desde una única interfaz gráfica, eliminando la necesidad de usar diferentes programas para cada sistema.

Algo que nos gustó mucho de DBeaver, es que podemos ver el diagrama de la base de datos, los datos de las tablas muy bien organizados, etc.





6.2 Función AUTO_INCREMENT ¿Cómo funciona?

Nos pareció sumamente interesante, como funciona AUTO_INCREMENT a nivel interno, así que buscando en internet, llegamos a las siguiente definición técnica: La función AUTO_INCREMENT no es una función matemática, ni una de busqueda, simplemente, **es un contador especial gestionado directamente en la memoria por el motor de almacenamiento de la base de datos.**

Cuando ejecutamos un INSERT, el motor de la base de datos pone un candado especial de nivel de tabla. Este candado es ultraligero y dura una fracción de milisegundos: solo el tiempo exacto que toma leer y sumar el número.

Con el candado puesto, el motor lee el valor actual del contador, el cual vive en la memoria RAM del servidor para acceso inmediato. Si el contador está en 5, el motor extrae ese 5 y lo sella en la fila que estás intentando insertar.

Antes de soltar el candado, el motor suma automáticamente 1 al contador en la memoria. Todo este proceso ocurre antes de que el disco duro siquiera empiece a escribir físicamente los datos de tu fila.

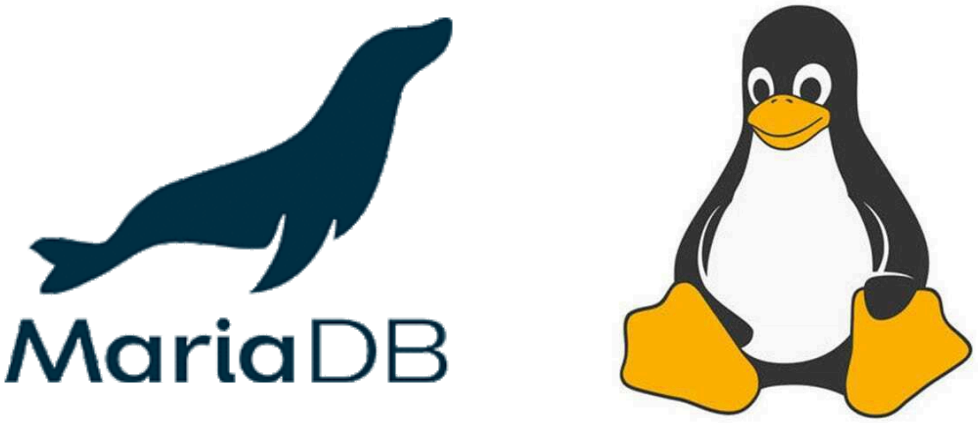
El motor quita el candado. Si otra consulta estaba esperando para hacer un INSERT, ahora puede tomar el número 6, sin importar si tu inserción original ya terminó de guardarse en el disco o no.

6.3 MariaDB

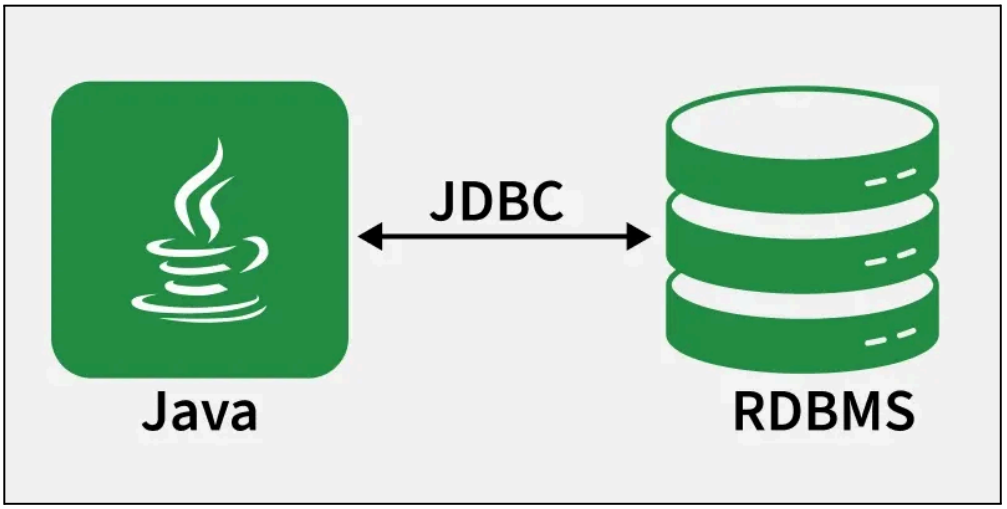
A la hora de desarrollar la base de datos, uno de los desarrolladores tiene una distribución basada en **Arch Linux**, y no encontró ningún repositorio con **MySQL.** Entonces, ese desarrollador utilizó **MariaDB** para el desarrollo de la base de datos. MariaDB es un

sistema de gestión de bases de datos relacionales (RDBMS) de código abierto y gratuito, licenciado bajo la GPLv2. Fue creado en 2009 por Michael Widenius, uno de los desarrolladores originales de MySQL, como una bifurcación (*fork*) de este para garantizar que el software permaneciera libre tras la adquisición de MySQL por parte de Oracle.

Luego de esto, nos dimos cuenta que la sintaxis de MariaDB y de MySQL son similares y funcionan exactamente igual.



7. Script en Java



Para hacer el script en Java que nos permitiera hacer consultas sobre nuestra Base de Datos lo que hicimos fue plantear una clase **Main** que funcione como un menú, el cual, mediante la selección de números en la terminal, permite ejecutar un método deseado.

Este mencionado menú lo hacemos mediante la clase **Scanner** de Java, la cual funciona como un input del mismo lenguaje. Le pedimos al usuario que ingrese un número determinando **el método que desee ejecutar**:

1. insertarProveedores(connection, scanner)
2. eliminarProveedor(connection, scanner)
3. listarProveedores(connection)
4. Salir del programa.

En todos pasamos **connection** como parámetro, ya que necesitamos hacerle referencia a nuestra función hacia la **conexión con la Base de Datos.**

7.1 insertarProveedores(Connection connection, Scanner scan)

Primero, establecemos un bloque try/catch/finally, esto tal que, si no podemos insertar algo a la base, nos tire un error en tiempo de ejecución. Primero, apagamos el autoguardado para iniciar la transacción, de tal manera que nosotros vamos a controlar cuándo guardar cambios (commit) y cuando no guardarlos (rollback).

Mediante la terminal, le pedimos al cliente que ingrese los respectivos datos del proveedor, los cuales son CUIT, Razón Social, Dirección, Teléfono Fijo y también pedimos Celular, ya que recordemos, proveedor tiene un atributo multivaluado y a este mismo lo tenemos que insertar en otra tabla.

Luego de esto, hacemos la primera consulta hacia nuestra base de datos, la consulta es:

INSERT INTO proveedor (cuit, razon_social, direccion, telefono_fijo) VALUES (?, ?, ?, ?)

Despues, instanciando un objeto **prepareStatement**, podemos ingresar parametros a VALUES (?, ?, ?, ?). Luego ejecutamos la consulta mediante .executeUpdate().

Una vez realizada la primera consulta, realizamos la segunda, la cual es para insertar los datos correspondientes en la tabla **mproveedor**.

INSERT INTO mproveedor (cuit, celular) VALUES (?, ?)

Una vez cargado los valores, ejecutamos la consulta y cerramos la consulta SQL y hacemos un .commit() para efectua la transaccion, de esta manera, hacemos permanentes los cambios en nuestra base de datos.

En caso de que algo falle (catch), revertimos la transaccion con rollback. Una vez haya terminado todo, con finally prendemos nuevamente el autoguardado y, en caso de que algo falle, mostramos el error mediante la consola.

7.2 eliminarProveedor(Connection connection, Scanner scan)

Para el método eliminarProveedor, lo que hacemos es pedirle un CUIT por consola a nuestro cliente, de esta manera, ese va a ser el CUIT del proveedor a eliminar.

Nos guardamos el CUIT en una variable, y ejecutamos la consulta:

DELETE FROM proveedor WHERE cuit = ?

Donde, simplemente, eliminamos los datos de aquel proveedor que tenga el mismo CUIT que el almacenado en **?**, el cual, va a ser el CUIT ingresado por nuestro cliente mediante la consola.

7.3 listarProveedores(Connection connection)

Para este método, lo que hacemos es ejecutar la consulta:

SELECT * FROM proveedor

En base a esta consulta, creamos un conjunto con todos los proveedores que nos de la consulta, mediante resultSet.

A este conjunto lo vamos a recorrer con un bucle while, el cual, tiene como condición que recorra el conjunto mientras resultSet.next() (mientras que haya un siguiente).

En el bucle, lo que hacemos que, mientras haya un siguiente, creamos una variable para cada campo de la tabla proveedor, obtenemos su valor mediante el método .getTipoDeDato() para luego imprimirlo por pantalla en un solo String.

8. Consultas SQL



A la hora de hacer las consultas SQL (DML), resolvimos las siguientes consultas:

Consulta 1: Obtener los modelos y números de talla de los zapatos que tienen la mayor cantidad vendida dentro de su respectivo modelo.

Consulta 2: Obtener el monto total que tiene asociado cada vendedor en 2025.

Consulta 3: Obtener todos los vendedores que no tuvieron **NINGÚN FRANCO** durante el mes de enero.

Y luego, planteamos estas consultas:

Consulta 4: Utilizando la subconsulta 2 listar los datos del vendedor con el monto de ventas asociado más alto.

Consulta 5: Selecciona el nombre y apellido de los vendedores que trabajan el dia Lunes.

Consulta 6: Selecciona modelos de zapatos junto a su color y talle cuyo precio supere los $100.000.

9. Conclusiones de la Etapa 1

A modo de conclusión, queremos cerrar esta presentación con un breve resumen de nuestras sensaciones y aprendizajes sobre esta primera entrega del Trabajo Práctico de esta materia. Esta primera fase contó con la realización de un diagrama de E/R y luego la tabulación del mismo. Fue interesante a modo de repaso de los conceptos que vimos en la primera etapa de la materia, y no nos presentó un desafío mayor por esa misma razón.

El principal aprendizaje que podemos llevarnos de esta etapa fue el trabajo en equipo y la comprensión de las narrativas desde distintas perspectivas grupales, porque al ser 5 personas en un mismo grupo, muchas veces las interpretaciones de representaciones particulares son distintas, y es importante la puesta en común para intercambiar las ideas y terminar llegando a un acuerdo.

Como dijimos previamente, las 2 tareas que tuvimos que hacer no nos presentaron una gran dificultad, pudimos comprender correctamente la narrativa presentada en el trabajo, y luego pudimos plasmarla en un diagrama que después tabulamos. Lo que sí fue de una ligera dificultad para el grupo fue la realización de este informe, porque en la carrera no estamos acostumbrados a documentar nuestros trabajos de esta manera. Aún así, pudimos dividir correctamente los roles y comprender la forma de realizar el informe aquí mismo presentado de la mejor manera y con todo nuestro proceso documentado.

10. Conclusiones de la Etapa 2

Para concluir la etapa 2, queremos cerrar la etapa con un resumen muy corto de lo que aprendimos. Consideramos que lo aprendido acerca de la implementación de tablas y sus respectivos datos nos va a ayudar tanto en la 2da instancia evaluativa de la asignatura como también en nuestro futuro profesional. En esta etapa se nos presenta un desafío mayor a comparación de la etapa anterior, ya que, tuvimos que decidir qué motor de SQL utilizar y en base a eso evaluar cómo planificar nuestra implementación de la base de datos teniendo en cuenta sintaxis, implementación de tablas, creación del trigger, etc.

Además del desafío presentado, nos encantó haber indagado en el uso de otras tecnologías de código abierto como lo son DBeaver y MariaDB.

11. Conclusiones Finales

Para cerrar este proyecto, entre los miembros del grupo, concluimos que fue un gran proyecto, nos gusto mucho la manera de arrancar desde simples diagramas hasta llegar a una base de datos funcional con su propio Script para manejar la misma, el cual, es extensible a muchas más herramientas de uso.

Frase que caracteriza a nuestra Base de Datos:

“Un gran poder, conlleva una gran responsabilidad”

- *Ben Parker.*